

Real-Time Vision System for Autonomous Ground Robots

Kareem Hassan, Samarth Vora, Murad Ibrahimov, Megan Sheehan
Purdue University
West Lafayette, Indiana, USA

Abstract—Autonomous ground robots require reliable environmental perception for navigation, obstacle avoidance, and real-time decision making. Traditional autonomous systems often rely on expensive and hardware-intensive sensor suites such as LiDAR, radar, and ultrasonic sensors. In this project, we developed a real-time monocular vision system capable of generating a 3D representation of the environment using a standard RGB webcam and neural-network-based depth estimation. Our system utilizes Intel-ISL’s MiDaS depth estimation model, OpenGL-based visualization, and DBSCAN clustering for obstacle detection. The project demonstrates how low-cost vision systems can approximate environmental understanding while reducing hardware dependencies for autonomous robotics applications.

I. INTRODUCTION AND MOTIVATION

Autonomous robots depend heavily on perception systems to understand and interact with their surroundings. Modern commercial systems, such as delivery robots and autonomous vehicles, often combine multiple sensors including LiDAR, radar, ultrasonic sensors, GPS, and cameras to construct reliable environmental maps. While effective, these systems can be expensive, computationally intensive, and difficult to integrate into smaller robotics platforms.

The motivation for this project originated from two senior design projects involving autonomous robotic systems. One project focused on an autonomous salt-spreading robot for sidewalks, while the other involved an autonomous race car platform. Both projects required obstacle detection and environmental awareness but faced practical limitations regarding sensor integration and hardware costs.

Initially, the team planned to integrate LiDAR sensors into the system. However, due to hardware accessibility constraints and integration challenges with the existing robotic platforms, the project shifted toward monocular depth estimation using computer vision techniques. The goal became developing a low-cost real-time vision system capable of:

- Reconstructing a 3D representation of the environment
- Detecting and clustering obstacles
- Providing real-time visualization for debugging and navigation
- Reducing dependence on specialized hardware

II. RELATED WORK / PRIOR ART

Modern autonomous systems heavily rely on multi-sensor fusion for perception. One example is the Starship delivery robot platform, which utilizes cameras, radar, ultrasonic sensors, and neural networks to maintain what the company refers

to as a “bubble of awareness” around the robot. These systems provide robust obstacle detection but require substantial hardware resources.

Our project was inspired by recent developments in monocular depth estimation research, particularly Intel-ISL’s MiDaS model. MiDaS is a deep learning framework that estimates relative inverse depth from a single RGB image. Unlike stereo vision or LiDAR systems, monocular depth estimation only requires a standard camera feed, making it significantly more affordable and easier to deploy.

The project also draws inspiration from point cloud visualization systems commonly used in robotics and SLAM pipelines. While traditional systems often use LiDAR-generated point clouds, our implementation instead generates pseudo-3D point clouds using estimated depth values from MiDaS.

For obstacle segmentation, the project utilizes the DBSCAN clustering algorithm. DBSCAN is commonly used in robotics and computer vision because it can identify arbitrarily shaped clusters while ignoring noisy outlier points.

III. TECHNICAL APPROACH

A. System Overview

The system consists of four primary components:

- 1) Camera input acquisition
- 2) Monocular depth estimation using MiDaS
- 3) Point cloud generation and obstacle clustering
- 4) OpenGL-based real-time visualization

The system pipeline is illustrated conceptually as:

RGB Camera → MiDaS Depth Estimation → Point Cloud
Generation → DBSCAN Clustering → OpenGL
Visualization

B. Camera Input and Multithreading

The system captures real-time frames from a webcam using OpenCV. To improve responsiveness and maintain real-time performance, the application uses multithreading. Separate threads are responsible for:

- Camera frame acquisition
- Depth estimation inference
- Vision processing and clustering

This architecture allows the rendering loop to remain responsive while the neural network performs depth estimation asynchronously.

C. Monocular Depth Estimation

The project uses the MiDaS DPT-Hybrid model loaded through PyTorch Hub. Incoming frames are resized and transformed before being passed through the neural network. The output is a relative inverse depth map.

The estimated depth values are normalized and converted into approximate metric depth values using configurable scaling and offset parameters. Additional filtering is performed using:

- Bilateral filtering
- Median blur smoothing
- Percentile normalization

These operations reduce noise and improve depth stability.

D. 3D Point Cloud Reconstruction

After depth estimation, the system converts image pixels into 3D coordinates using a pinhole camera projection model.

Given image coordinates (u, v) and estimated depth z , the 3D coordinates are computed as:

$$x = \frac{(u - c_x)}{f_x} z$$
$$y = -\frac{(v - c_y)}{f_x} z$$

where:

- f_x is the focal length approximation
- (c_x, c_y) is the image center

The generated points are classified into:

- Ground points
- Boundary points
- Obstacle points

Classification is based on spatial thresholds and geometric constraints.

E. Obstacle Detection Using DBSCAN

Obstacle points are clustered using the DBSCAN algorithm from Scikit-learn.

DBSCAN parameters:

- $\epsilon = 0.5$
- Minimum points = 10

The clustering algorithm groups nearby points into obstacle regions while rejecting sparse noise points.

To avoid incorrectly classifying walls or floors as obstacles, a custom “Wall Rejection Filter” was implemented. Clusters exceeding predefined width or height thresholds are discarded. Specifically:

- Width > 2.5 meters $\Rightarrow$ reject
- Height > 2.5 meters $\Rightarrow$ reject

This significantly improved obstacle detection quality.

F. Visualization Pipeline

Visualization is implemented using PyGame and OpenGL. The renderer displays:

- A 3D point cloud representation
- Real-time obstacle bounding boxes
- Camera feed inset
- Depth map inset
- Performance statistics

Users can rotate the scene interactively using mouse controls to inspect the reconstructed environment from different perspectives.

IV. TIMELINE AND DEVELOPMENT PROCESS

A. Week 1

- Setup Python environment
- Configure OpenCV and OpenGL pipeline
- Display live camera feed
- Create basic coordinate system

B. Week 2

- Implement edge-based obstacle detection
- Render 3D points and basic geometry
- Develop OpenGL visualization pipeline

C. Week 3

- Attempt LiDAR integration
- Transition from LiDAR to MiDaS depth estimation
- Integrate neural-network-based depth estimation

D. Week 4

- Implement DBSCAN clustering
- Add wall rejection filtering
- Optimize performance using multithreading
- Conduct testing on robotics platforms

Responsibilities were divided among team members based on interests and expertise. Team members worked collaboratively on visualization, depth estimation, system integration, and testing.

V. CURRENT RESULTS

The current system successfully:

- Generates real-time monocular depth maps
- Reconstructs pseudo-3D environments
- Detects obstacles using point cloud clustering
- Displays interactive 3D visualizations

The OpenGL visualization demonstrates the conversion from a standard RGB camera feed into a navigable point cloud representation. Bounding boxes generated using DBSCAN effectively identify distinct obstacles in the environment.

The system achieves real-time performance through multithreading and GPU acceleration when CUDA is available.

Qualitatively, the MiDaS model performs well in indoor environments and provides visually coherent depth estimates. However, depth accuracy remains relative rather than fully metric.

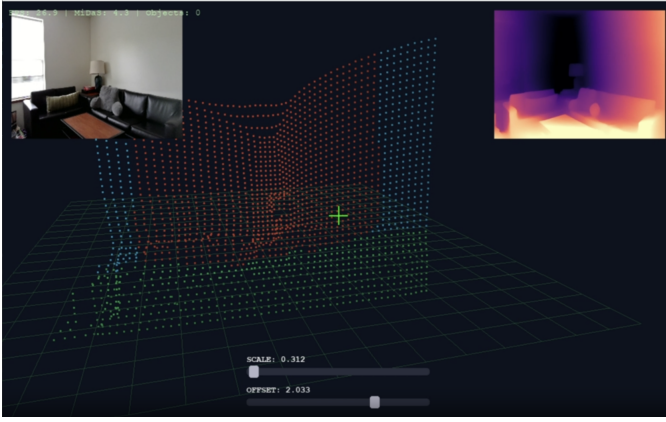


Fig. 1. Example OpenGL visualization of reconstructed environment and obstacle detection.

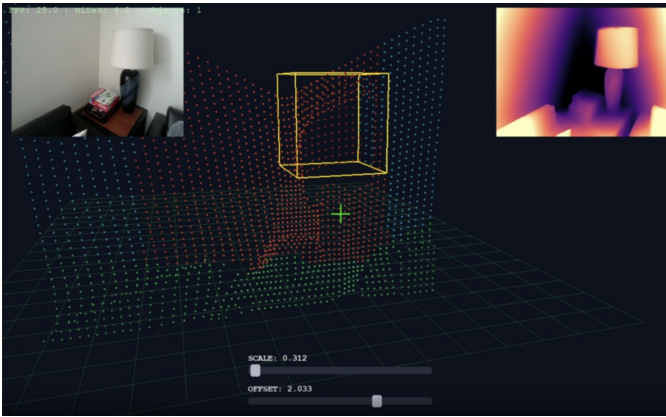


Fig. 2. Example OpenGL visualization of object detection.

VI. VIDEO AND CODE LINKS

Project Video: [Demo Video](#)

GitHub Repository:

<https://github.com/Murad777-phy/Graphics-Project.git>

VII. LIMITATIONS AND FUTURE WORK

Although the project demonstrates promising results, several limitations remain.

First, monocular depth estimation provides relative depth rather than true metric depth. As a result, distance measurements can vary depending on scene composition and lighting conditions.

Second, the system currently relies on manually tuned parameters for clustering and filtering. More adaptive techniques could improve robustness across different environments.

Additional future improvements include:

- Integration of stereo vision or sensor fusion
- Improved camera calibration for metric scaling
- Better temporal filtering and object tracking
- Quantitative performance evaluation
- User studies and comparison against LiDAR-based methods

- Deployment on embedded robotics hardware

Further research could also explore combining monocular depth estimation with SLAM techniques for simultaneous localization and mapping.

VIII. CONCLUSION

This project demonstrates the feasibility of using monocular depth estimation and computer vision techniques to create a low-cost real-time perception system for autonomous robots. By combining MiDaS depth estimation, OpenGL visualization, and DBSCAN clustering, the system can reconstruct 3D environments and detect obstacles using only a standard RGB camera.

While the system does not yet match the reliability of industrial multi-sensor platforms, it provides a promising foundation for future research into affordable and lightweight autonomous perception systems.

REFERENCES

- [1] R. Ranftl et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer," IEEE Transactions on Pattern Analysis and Machine Intelligence, 2020.
- [2] Starship Technologies, "Our Robots," Available: <https://www.starship.xyz/our-robots/>
- [3] M. Ester, H. Kriegel, J. Sander, and X. Xu, "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," KDD, 1996.
- [4] OpenCV Library, Available: <https://opencv.org/>
- [5] PyTorch Documentation, Available: <https://pytorch.org/>